PS1_1 report

运行结果：

```
[1]: #PS1_1

# 定义一个函数将a,b,c按从大到小进行排序
import random

def Print_values(a,b,c):

    # 判断a是否大于b
    if a > b:
        # 如果 a > b，继续判断 b > c
        if b > c:
            # 如果 b > c，按照 a、b、c 的顺序输出
            print(a,b,c)
        else:
            # 如果 a <= b，继续判断 a > c
            if a > c:
                # 如果 a > c，按照 a、c、b 的顺序输出
                print(a,c,b)
            else:
                # 如果 a <= c，按照 c、a、b 的顺序输出
                print(c,a,b)
    else:
        # 如果 a <= b，继续判断 b > c
        if b > c:
            # 如果 b > c，则不输出结果
            pass
        # 如果 b <= c，按照 c、b、a的顺序输出
        else:
            print(c,b,a)

print("开始测试随机值")
a = random.randint(1,100)
b = random.randint(1,100)
c = random.randint(1,100)

Print_values(a,b,c)
```

```
开始测试随机值
99 91 63
```

测试结果：

```
开始测试随机值
99 91 63
```

PS1_2 report

运行结果：

2.1

•[2]:

```
#PS1_2

#2.1
import random

# 建立 M1 空矩阵
M1 = []

# 创建5行
for i in range(5):

    # 创建1行空列表
    row = []

    # 每行添加10个随机数值
    for j in range(10):

        number = random.randint(0,50)

        # 将随机数字添加到行中
        row.append(number)

    # 将行添加到 M1 中
    M1.append(row)

# 将 M1 矩阵结果输出
print(M1)


# 建立 M2(10,5) 矩阵
import random
M2 = []
for i in range(10):
    row = []
    for j in range(5):
        number = random.randint(0,50)
        row.append(number)
    M2.append(row)

print(M2)
```

2.2

```python
# 2.2
def Matrix_multip(M1,M2):

    # 定义结果为result
    result = []

    # 使用 for 循环，重复生成至第5行
    for i in range(5):

        row = []

        # 使用 for 循环，重复生成至第5位元素
        for j in range(5):

            # 将当前矩阵位置的计算结果定义为 total,初始值为0
            total = 0

            # M1 有10列， M2 有10行，k 对应M1当前行的第 k 个元素 和M2当前列的第 k 个元素
            for k in range(10):

                # 根据计算规律可得， M1ij 中 j = k, M2ij 中 i = k
                total = total + M1[i][k]* M2[k][j]

            row.append(total)

        result.append(row)

    return result

result = Matrix_multip(M1,M2)

print(result)
```

测试结果：

```
[[49, 17, 3, 41, 21, 30, 27, 10, 30, 11], [11, 8, 16, 25, 46, 35, 42, 2, 44, 27], [20, 42, 5, 18, 49, 16, 19, 1, 10, 35], [45, 16, 17, 49, 50, 28, 22,
37, 40, 34], [3, 35, 43, 40, 25, 35, 31, 43, 30, 16]]
[[38, 4, 40, 41, 48], [24, 45, 4, 14, 33], [40, 20, 1, 43, 16], [18, 29, 10, 9, 4], [31, 2, 39, 26, 10], [17, 5, 22, 44, 31], [45, 33, 23, 34, 28], [8,
34, 14, 42, 43], [45, 13, 45, 48, 3], [1, 20, 25, 25, 17]]
[[6945, 4243, 6306, 7664, 5728], [7634, 4282, 6951, 8511, 4546], [5431, 4261, 5192, 5806, 4684], [8802, 6085, 8611, 10845, 7429], [7869, 7027, 5513, 97
12, 6561]]
```

PS1_3 report

运行结果：

```python
#PS1_3

def Pascal_triangle(k):

    # 创建1个包含1的列表作为 Pascal_triangle 的第一行
    row = [1]#第 0 行
    # 使用 for 循环重复生成至 k 行,
    for i in range(k):

        #每次循环前创建一行包含1的新列表
        new_row = [1]

        # 使用 for 循环计算至上一行（元素数-1） 的次数
        for j in range(len(row)-1):

            # 把上一行中两个相邻的元素相加所得的结果，添加至新的一行
            new_row.append(row[j] + row[j+1])

        # 在新一行的末尾加1
        new_row.append(1)

        # 将新生成的结果返还至 row
        row = new_row

    print(row)


Pascal_triangle(100)
Pascal_triangle(200)
```

测试结果：

Pascal_triangle(100)

```
[1, 100, 4950, 161700, 3921225, 75287520, 1192052400, 16007560800, 186087894300, 1902231808400, 17310309456440, 141629804643600, 1050421051106700, 7110
542499799200, 44186942677323600, 253338471349988640, 1345860629046814650, 6650134872937201800, 30664510802988208300, 132341572939212267400, 53598337040
3809682970, 204184141106213212560 00, 733206688517765626 9200, 2486527030625466039 1200, 7977607556590036875 5100, 2425192697203371210 15504, 699574816500972
464467800, 191735320078044305076 3600, 499881370203472652520 5100, 124108478119482865453 36800, 293723398216109448239 63760, 663246383068634237 96047200, 14
301250134917425756022 6775, 294692427022540894366527900, 580717429720889409486981450, 109506715318796286846116 5020, 197720458214493298944 3770175, 342002
954749393814390273 7600, 567004898663468692278 6117600, 901392403003463049263 4340800, 137462341458028115012673 69720, 201164402133699680506351 75200, 28258
808871162574166368460400, 381165328959867279453342 02400, 493782357970737157473647 62200, 614484712141361679596720592960, 734709981908149973439 05056800, 8
441348728306403950150793 7600, 932065588750498769495816 81100, 989130828878080326811887 22800, 100891344545564193334812497256, 989130828878080326811887228
00, 932065588750498769495816 81100, 844134872830640395015079 37600, 734709981908149973439 05056800, 614484712141361795967205929 60, 493782357970737157473647
62200, 381165328959867279453342 02400, 282588088711625741663684 60400, 201164402133699680506351 75200, 137462341458028115012673 69720, 901392403003463049 2
634340800, 567004898663468692278 6117600, 342002954749393814390273 7600, 197720458214493298944 3770175, 109506715318796288646116 5020, 580717429720889409 48
6981450, 294692427022540894366527900, 143012501349174257560226775, 663246383068634237 96047200, 293723398216109448239 63760, 124108478119482865453 36800,
499881370203472652520 5100, 191735320078044305076 3600, 699574816500972464467800, 242519269720337121015504, 797760755659003687 55100, 248652703062546603 91
200, 733206688517765626 9200, 204184141106213212560 0, 535983370403809682970, 132341572939212267400, 306645108029882083 00, 665013487293720180 0, 134586062
9046814650, 253338471349988640, 44186942677323600, 7110542499799200, 1050421051106700, 141629804643600, 17310309456440, 1902231808400, 186087894300, 16
007560800, 1192052400, 75287520, 3921225, 161700, 4950, 100, 1]
```

Pascal_triangle(200)

[1, 200, 19900, 1313400, 64684950, 2535650040, 82408626300, 2283896214600, 55098996177225, 1175445251780800, 22451004309013280, 387790074428411200, 610
7693672247476400, 88326646952501966400, 11797916414369905512 00, 1462941635381868283 4880, 1691526265910285 20278300, 1830828428985249866541600, 186134223
6135004030983960 0, 1782969931455635440 20568800, 1613587787967350073386147640, 1383075246829157205759551200, 1125320314465541544686818348400, 8709000694
5594084762669852240 0, 6422888012237563751246901602700, 4521713160615244880877818728300 8, 3043460781183337900590839528664 00, 1961341392318150914918743 6
2916800, 1211828788825143352957479517087880 0, 7187398333721539886506430239279840 0, 4096817050221277735308665236389508 80, 2246641608185861983878945452 21
3601600, 1186507599323158360236068066950308345 0, 6040402323826988015747255613565206120 0, 2966903494350314701852328492545263006 00, 1407159943034720687 16
4247227892896168560, 6449483072242469816169466461175774105900, 2858689794183148783383222971980613387480 0, 1226227464346982241293329853770631539 9800, 5
0935602365182339253722932387395463636840 0, 2050157995198589154962348028592667411382810, 8000616566628640604731114257922604532225600, 3028804843080842 51
4648207540499271715771120 0, 1112909686527379342591666956745 1592863000427200, 3971064108745421745156629822918865442992516 00, 1376635557698412871654298 3 86
1187335357073888 0, 4638663292244652067530787880105225430510098400, 1519902440437609400850513475608946204890542880 0, 4844689028894879965211011703503 51 60
280886054300, 1502842311004126030024640365168437625039730664 00, 4538583779232460610674413902808681627619986605 28, 1334877582127194297257180559649612243417643119200, 4538583779232460610674413902808681627619986605 28, 15
0284231100412603002464036516843762503973066400, 4844689028894879965211011703503516028088605430 0, 1519902440437609400850513475608946204890542880 0, 46386
6329224465206753078788010522543051009840 0, 1376635557698412871654298338611873353570738880, 3971064108745421745156629822918865442992516 00, 1112909686527
3793425916669567415928630042720 0, 3028804843080842514648207540499271715771120 0, 8000616566628640604731114257922604532225600, 2050157995198589154962348 0
2859266741138281 0, 5093560236518233925372293238739546363684 00, 1226227464346982241293329853770631539 9800, 2858689794183148783383222971980613387480 0, 6
4494830722424698161694664611757744105900, 1407159943034720687164247227892896168560, 2966903494350314701852328492545263006 00, 6040402323826988015747255 61
35652061200, 1186507599323158360236068066950308345 0, 2246641608185861983878945452213601600, 4096817050221277735308665236389508 80, 7187398333721539886 50
64302392798400, 1211828788825143352957479517087880 0, 1961341392318151091491874362916800, 3043460781183337900590839528664 00, 4521713160615244880877818 72
83008, 6422888012237563751246901602700, 8709000694559408476266985224 00, 1125320314465541544686818348400, 1383075246829157205759551200, 1613587787967350
073386147640, 1782969931455635440205688 00, 1861342236135004030983960 0, 1830828428985249866541600, 1691526265910285 20278300, 1462941635381868283 4880, 11
7979164143699055120 0, 8832664695250196 6400, 6107693672247476400, 387790074428411200, 22451004309013280, 1175445251780800, 55098996177225, 228389621460
0, 82408626300, 2535650040, 64684950, 1313400, 19900, 200, 1]

PS1_4 report

运行结果：

```python
#PS1_4
# 设目标金额参数为x
import random
x = random.randint(1,100)

print("Target amount",x)

def Least_moves(x):

    # 初始金额为1
    money = 1

    # 如果目标金额是1，程序停止
    if x == 1:
        return 0

    else:
        moves = Least_moves(x - 1) + 1

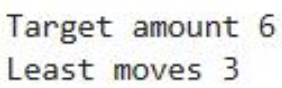
        # 判断条件
        if x % 2 == 0:
            moves = min(moves, Least_moves(x // 2)+1)

        # 将计算的最少步数返回 moves
        return moves

print("Least moves", Least_moves(x))
```

```
Target amount 6
Least moves 3
```

测试结果：



PS1_5 report

运行结果：
我的同学边越给我讲解了如何利用循环遍历所有可能的表达式，以及通过三进制表示不同的选择

```python
# PS1_5

def Find_expression(target):

    count = 0

    # 我的同学 Bian Yue 给我讲解了如何利用循环遍历所有可能的表达式，以及通过三进制表示不同的选择
    # 在1-9里面有8个空位来填写 + 或 - 或者数字链接，故有3^8个组合，将其全部遍历
    for i in range(3**8):
        expression = "1"
        number = i

        for digit in range(2,10):

            # 每个数字的三进制恰好表示一种可能
            choice = number % 3
            number = number // 3

            #通过基数转换算法提取出对应位的数字，进而转换为+ 或- 或者数字链接
            if choice == 0:
                expression = expression + str(digit)

            elif choice == 1:
                expression = expression + "+" +str(digit)

            else:
                expression = expression + "-" + str(digit)

        if eval(expression) == target:
            print(expression, "=", target)
            count = count + 1
    #返回满足条件的表达式数量
    return count


#5.1
#调用函数
Find_expression(50)
```

5.2

```python
#5.2
#设置列表保存满足条件的表达式的数量
Total_solutions = []

#从1到100 调用函数
for i in range(1,101):
    count = Find_expression(i)

    #把函数的返回值即满足条件的表达式的数量存入列表
    Total_solutions.append(count)
```

测试结果：

Find_expression(50)

```
1-2+34-5-67+89 = 50
1+2+3+4-56+7+89 = 50
1-2-34-5-6+7+89 = 50
1+2-34+5-6-7+89 = 50
12+3+4-56+78+9 = 50
1-2+3-45+6+78+9 = 50
1-23-4-5-6+78+9 = 50
1+2+34-5-6+7+8+9 = 50
1-2-3+4+56-7-8+9 = 50
1+2+34-56+78-9 = 50
1-23+4+5-6+78-9 = 50
1-2-3-4-5-6+78-9 = 50
1-2+34+5+6+7+8-9 = 50
1+2+3-4+56-7+8-9 = 50
12-3-4-5+67-8-9 = 50
1+2-3+4+56+7-8-9 = 50
12-3+45+6+7-8-9 = 50
```

```
Total_solutions = [26, 11, 18, 8, 21, 12, 17, 8, 22, 12, 21, 11, 16, 15, 20, 8, 17, 11, 20, 15, 16, 11, 23, 18, 13, 14, 21, 15, 19, 17, 14, 19, 19, 7,
14, 19, 19, 17, 18, 16, 17, 18, 10, 15, 26, 18, 15, 16, 12, 17, 19, 9, 17, 21, 16, 13, 14, 16, 17, 17, 11, 13, 22, 14, 13, 15, 15, 15, 17, 7, 14, 17, 1
5, 12, 13, 14, 14, 14, 10, 9, 19, 12, 13, 13, 12, 11, 12, 6, 12, 14, 16, 13, 11, 11, 10, 11, 7, 9, 17, 11]
Maximum number of solutions: 26
Minimum number of solutions: 6
Numbers with maximum solutions:
1
45
Numbers with minimum solutions:
88
```

Maximum number of solutions ： 1、45

Minimum number of solutions ： 88